

LLVM/C++, program analysis, SSA, and compiler/runtime engineering. At MCST I implemented a LICM pass and a regression harness that compares semantics before and after optimization. I created UniversalToolchain/Wist2 with callable-first SSA, interpreter/CIL backends, and verifier-gated transformations; a talk on the platform architecture was accepted for presentation at LangDev 2026.

MCST · LLVM/C++

LICM pass + semantic before/after regression harness

1st of 49 · 104/104

PS-form memory-dependence analysis; the only 5.0/5.0 score

LangDev 2026

Accepted presentation on UniversalToolchain/Wist2 language architecture

EXPERIENCE

- July 1 — August 31, 2026

MCST — compiler engineering intern

 - Implemented an LLVM LICM pass in C++ with conservative gates for side effects, memory reads/writes, and unsafe speculative execution; only loop-invariant instructions are hoisted to a preheader.
 - Built a Python regression harness: baseline mem2reg + loop-simplify versus optimized + LICM, semantic parity through lli, expected CHANGE/NO_CHANGE cases, and IR validation with -verify-each.
- 2026

PS-form Memory Dependence Analyzer

 - Implemented conservative overlap analysis for parametric memory accesses across loop iterations; ranked 1st of 49 with the only 5.0/5.0 score and 104/104 official tests.
 - Built a post-selection exact oracle for small domains, randomized/metamorphic tests, and reproducible wrong-answer/time-limit counterexamples.
- 2024 — present

UniversalToolchain / Wist2 — creator and primary developer

 - Created a modular compiler/runtime platform with Bytecode/AIR, callable-first SSA, optimizations, interpreter execution, and a CIL backend.
 - Develop verifier-gated transformations, typed compiler facts/effects, and interpreter/CIL parity checks so semantic violations fail before execution.

PROJECTS

- UniversalToolchain / Wist2

Semantic verification and backend-parity checks act as fail-closed regression gates, with verification records tied to exact revision/run identities.
- PS-form Memory Dependence Analyzer

Ranked 1st of 49 with the only 5.0/5.0 score and 104/104 official tests; the post-selection harness preserves reproducible wrong-answer/time-limit counterexamples.
- x86-64 Codegen & Register Allocation Lab

A measurable pipeline with spill/reload metrics, code size, and reproducible result comparison.

SKILLS

- LLVM / C++

C++23, LLVM pass plugins, LICM, LoopInfo, CMake, clang/opt/lli, -verify-each
- Program analysis

CFG, dominance, SSA, dataflow, memory dependence, side effects, speculative safety
- Compiler / runtime

Bytecode/AIR, callable-first SSA, optimizations, interpreter, CIL backend, typed semantics
- Verification

semantic before/after regression, exact oracles, differential/metamorphic testing, sanitizers

EDUCATION

Software Engineering student at HSE University

RECOGNITION

- The Program Committee accepted “Build the Language, Then Make the Abstractions Disappear: Extensible Programming on .NET” for presentation on UniversalToolchain/Wist2 architecture in Torremolinos, Spain.

Moscow / remote